

Metropolis and HMC algorithms

Paolo Garimberti

Contents

1	Theoretical background	2
1.1	Markov chains	2
1.2	Scalar field theory on the lattice	5
1.3	Numerical simulations and error analysis	7
2	Algorithms	10
2.1	Metropolis	10
2.2	Hybrid Monte Carlo	11
3	Numerical results	14
3.1	The ϕ^4 -theory: Metropolis implementation	15
3.2	The ϕ^4 -theory: Hybrid Monte Carlo implementation	23
4	SSB in ϕ^4-theory	30
4.1	Data analysis	31

1 Theoretical background

In order to provide a context, I have first introduced the theoretical background upon which my numerical studies are based. Therefore, I will give a quick review of the most important concepts in Markov Chains, Scalar Field Theory on the lattice and Error Analysis. The part which deals with Numerical Results starts at pag.14.

1.1 Markov chains

Discrete stochastic processes

A *discrete stochastic process* is a family of random variables $\{X(n)\}_{n \in \mathbb{N}_0}$ indexed by the integers. In this context, the index n assumes the meaning of *time*, and a sequence of $X(n)$ is considered along this time, describing the stochastic evolution of the system. Each random variable $X(n)$ can take a value out of a set of possible values $\{E_k\}_{k \in S}$, where $S \subset \mathbb{N}_0$ is countable and finite.

We are interested in a particular kind of discrete, *time-homogeneous* - i.e. conditional probabilities P_{jk} do not depend on the Markovian time n - stochastic processes: *Markov chains*. To be precise, a discrete stochastic process is a *Markov chain* if the probability of obtaining the sequence of states $E_{i_0}, E_{i_1}, \dots, E_{i_n}$ is given by

$$P(X(0) = E_{i_0}, X(1) = E_{i_1}, \dots, X(n) = E_{i_n}) = a_{i_0} P_{i_0 i_1} P_{i_1 i_2} \dots P_{i_{n-1} i_n} \quad (1)$$

where a_{i_0} is the probability to obtain E_{i_0} for the first value of the sequence, and P_{jk} with $j, k \in S$ is the *conditional probability* that the system makes a transition to the state E_k if it is currently in the state E_j .

Moreover, since a_{i_0} and P_{jk} for each pair $j, k \in S$ are probability distributions, it can be easily shown that $P(E_{i_0}, E_{i_1}, \dots, E_{i_n})$ is itself a probability distribution on the state space of $n + 1$ replicas of the system.

Transition probability matrix

The transition probabilities associated to a given Markov chain can be collected into a *transition probability matrix*:

$$\mathbb{P} = \begin{bmatrix} P_{11} & P_{12} & P_{13} & \dots \\ P_{21} & P_{22} & P_{23} & \dots \\ P_{31} & P_{32} & P_{33} & \dots \\ \dots & \dots & \dots & \dots \end{bmatrix} \quad (2)$$

which, along with the probability distribution of the initial state a_{i_0} , completely determines the Markov chain. It is clear that, by definition, $\mathbb{P}$ is a square matrix which - in the case of discrete Markov chains with finite state space - contains a finite number of elements.

Let's now denote with $P_{jk}^{(n)}$ the probability that the system is in state E_k ,

starting from E_j , after n steps. It holds the *Chapman-Kolmogorov* relation:

$$P_{jk}^{(n+m)} = \sum_l P_{jl}^{(n)} P_{lk}^{(m)}. \quad (3)$$

As before, the n -step transition probabilities $P_{jk}^{(n)}$ can be organized in a matrix, whose elements are naturally defined as

$$\mathbb{P}_{jk}^{(n)} \equiv P_{jk}^{(n)}. \quad (4)$$

The n -step transition probability matrix $\mathbb{P}^{(n)}$ inherits from the single-step transition probability matrix $\mathbb{P}$ the properties

$$\mathbb{P}^{(n)} = (\mathbb{P})^n, \quad (5)$$

$$\mathbb{P}^{(n+1)} = \mathbb{P}(\mathbb{P})^n, \quad (6)$$

$$\mathbb{P}^{(n+m)} = (\mathbb{P})^n (\mathbb{P})^m. \quad (7)$$

Classification of states

There exist different kind of states which can be associated to a given Markov chain. Classification is useful to prove important results connected with the theory of Markov chain.

In order to understand the classification, a preliminary definition has to be given. For this purpose, let's define the probability $f_{jk}^{(n)}$ that the system gets *for the first time* to the state E_k , starting from E_j , after n steps (and put $f_{jk}^{(0)} = 0$ by convention).

The probability that, starting from E_j , the state E_k will be eventually reached can be written in term of the $f_{jk}^{(n)}$ as

$$f_{jk} = \sum_{n=0}^{\infty} f_{jk}^{(n)} \leq 1 \quad (8)$$

and the inequality follows since f_{jk} is a probability. When $f_{jk} = 1$, $f_{jk}^{(n)}$ can be interpreted as a probability distribution in the variable n (in this case, n is regarded as a stochastic variable itself, distributed with $f_{jk}^{(n)}$). A measure of the average number of steps for the system to revisit a given state is provided by the *mean return time*

$$\mu_j = \sum_{n=1}^{\infty} n f_{jj}^{(n)}. \quad (9)$$

Given these definition, we can proceed with the classification of states.

- A state E_j is *recurrent* if $f_{jj} = 1$.
- A state E_j is *transient* if $f_{jj} < 1$.

- A recurrent state E_j is *null* if its average return time $\mu_j = \infty$.
- A state E_j is *periodic* with period $t > 1$ if $P_{jj}^{(n)} = 0$ except for $n = mt$, where t is the largest divisor of those n values. A state which is not periodic is said to be *aperiodic*.
- The state E_j is *ergodic* if it is recurrent, aperiodic and non-null ($\mu_j < \infty$).

Equilibrium probability

We are interested in the limit of an infinitely long Markov chain. In fact, it can be proved that - under certain conditions - an equilibrium probability for the Markov chain exists in this limit. To be more specific, the main result can be stated as follows: for an *irreducible* Markov chain, composed only of *ergodic* states, the limit

$$\Pi_j = \lim_{n \rightarrow \infty} P_{ij}^{(n)} \quad (10)$$

exists and it is unique, independent on the state i . Furthermore, the following conditions hold:

$$\sum_i \Pi_i = 1 \quad \text{and} \quad \Pi_j = \sum_i \Pi_i P_{ij} \quad (11)$$

Detailed balance condition

Let's consider a probability distribution

$$P_i = \frac{R_i}{\sum_j R_j} \quad R_i \geq 0, \quad \forall i. \quad (12)$$

Given an *ergodic* Markov chain with transition probabilities P_{ij} , a sufficient condition that

$$\lim_{n \rightarrow \infty} P_{ij}^{(n)} = P_j \quad \forall i, \quad (13)$$

is the so-called *detailed balance condition*, which reads

$$R_i P_{ij} = R_j P_{ji}. \quad (14)$$

This condition is extremely important since it provides a general way to build a Markov chain whose equilibrium distribution is the one desired, even if there is still some freedom left on how to do it. A concrete example is the *Metropolis algorithm*, to be presented later.

1.2 Scalar field theory on the lattice

Quantum Field Theory: a quick review

In this section, we briefly review some results in Quantum Field Theory, especially about *Path Integral formulation*, in order to focus on non-perturbative features of a simple quantum field theory - the scalar field theory ϕ^4 - using numerical methods.

For our purpose, let's consider a self-interacting Klein-Gordon field in $d + 1$ dimensions, whose dynamics is governed by the action

$$\mathcal{S} = \int dt d\vec{x} \mathcal{L}(\phi(t, \vec{x}), \partial_\mu \phi(t, \vec{x})), \quad \vec{x} = (x_1, x_2, \dots, x_d) \quad (15)$$

with the lagrangian density

$$\mathcal{L} = \frac{1}{2} \partial_\mu \phi \partial^\mu \phi - \frac{m^2}{2} \phi^2 - V(\phi), \quad g_{\mu\nu} = \text{diag}(1, -1, \dots, -1) \quad (16)$$

where the potential function $V(\phi)$ depends on the fields only (not on its derivatives).

As usual, one introduces the *Hamiltonian* by a Legendre transformation and defines the *conjugate momenta* of the field, $\Pi(t, \vec{x})$. The quantum features of the theory emerge once the canonical quantization prescriptions have been applied, i.e. promoting the fields ϕ and Π to operator-valued distributions $\hat{\phi}$ and $\hat{\Pi}$, and imposing the well-known commutation relations. To complete the procedure of quantization, however, the theory needs to be regularized and renormalized. We consider here the *lattice regularization*, the only known way to regularize a QFT non-perturbatively. Moreover, the introduction of the lattice provides an efficient way to implement numerical methods for studying the theory.

The lattice

Lattice theory is defined by introducing a finite d -dimensional lattice, given by the set of points

$$\Lambda_d = \{\vec{x} \mid \vec{x} = a\vec{n}, \text{ with } n_i = 0, \dots, N_i - 1 \text{ and } i = 1, \dots, d\}, \quad (17)$$

where a is the *lattice spacing*. The lattice is finite in all directions, with an extent $L_i = a N_i$. This allows to deal with a finite number of degrees of freedom (the field is defined only on the lattice points), which necessary in order to study the theory numerically. On top of that, due to the finiteness of the lattice, some boundary conditions on the lattice fields have to be imposed; for example, a convenient choice are *periodic boundary conditions*:

$$\hat{\phi}(\vec{x} + \hat{j}L_j) = \hat{\phi}(\vec{x}), \quad \hat{\Pi}(\vec{x} + \hat{j}L_j) = \hat{\Pi}(\vec{x}), \quad j = 1, 2, \dots, d. \quad (18)$$

Having a finite number of degrees of freedom, the system becomes equivalent to a conventional quantum mechanical system and the procedure of quantization is analogue. However, we will focus now on another way to quantize the system, through the *Path Integral*.

Path Integral formulation

The canonical formulation of a QFT is not very practical, and it is more convenient to introduce the *Path Integral formulation* (PI), which provides an efficient way to define the theory non-perturbatively and allows to implement numerical schemes in order to study it.

In order to obtain the PI formula, first one has to introduce a basis of states of the Hilbert space, eigenstates of the field $\hat{\phi}(\vec{x})$ and of the momenta $\hat{\Pi}(\vec{x})$:

$$\hat{\phi}(\vec{x})|\phi\rangle = \phi(\vec{x})|\phi\rangle, \quad \hat{\Pi}(\vec{x})|\Pi\rangle = \Pi(\vec{x})|\Pi\rangle, \quad (19)$$

where $\phi(\vec{x})$ and $\Pi(\vec{x})$ are now $\mathbb{C}$ -valued functions; in the PI formulation, the quantities of interest will be expressed in terms of conventional functions and not operators. The orthonormality and completeness of the basis will be used to obtain the path-integral of the theory. A key quantity to be considered in order to do that is the thermal partition function Z of the corresponding quantum mechanical system, which allows to express the expectation value of a generic function of quantum fields - let's call it $\hat{\mathcal{O}} \equiv \mathcal{O}(\hat{\phi}(\vec{x}))$ as

$$\langle \mathcal{O} \rangle = \frac{1}{Z} \text{tr}[e^{-\beta \hat{H}} \mathcal{O}]. \quad (20)$$

Recall, in fact, that a *Wick rotation* connects statistical mechanics to quantum mechanics, by replacing $\beta \equiv 1/k_B T$ with imaginary time $it/\hbar$. Therefore, the partition function plays an important role in order to compute expectation values of observables. It can be shown that both Z and $\langle \mathcal{O} \rangle$ admit a PI representation. Without going through all the steps, the PI representation of the expectation value of an observable can be written as

$$\langle \hat{\mathcal{O}} \rangle \approx \frac{1}{Z_a} \int D\phi e^{-S_E(\phi)} \mathcal{O}(\phi(x)), \quad (21)$$

where the symbol $\approx$ means that the formula is approximated because we are taking a finite value for N_0 ; in the previous formula, Z_a is the partition function for a discrete lattice spacing a . Of course, the physical predictions are obtained in the *infinite-volume limit*, i.e. $L_\mu \rightarrow \infty \forall \mu$ and $a \rightarrow 0$ (*continuum limit*), which removes regularization, after the theory has been renormalized. For numerical investigation, we will take the infinite-volume limit maintaining finite lattice spacing. By taking both limits, we can formally write

$$\langle \hat{\mathcal{O}} \rangle = \frac{1}{\mathcal{Z}} \int D\phi e^{-S_E(\phi)} \mathcal{O}(\phi(x)), \quad (22)$$

where $\mathcal{Z} = \int [D\phi] e^{-S_E(\phi)}$ is the *Euclidean partition function*, $[D\phi]$ is a measure over the space of functions ϕ and S_E is the *Euclidean action*, defined as

$$S_E(\phi) = \int d^{d+1} \left[\sum_{\mu=0}^d \frac{1}{2} \partial_\mu \phi(x) \partial_\mu \phi(x) + \frac{m^2}{2} \phi(x)^2 + V(\phi(x)) \right]; \quad (23)$$

the Euclidean theory is one-one correspondence with Minkowski theory. However, the Euclidean version is much more useful from a practical point of view. In fact, the factor $e^{-S_E(\phi)}/\mathcal{Z}$ can be seen as a probability distribution; thus, after defining the theory on the lattice, an importance sampling Monte Carlo can be adopted in order to compute path-integrals.

1.3 Numerical simulations and error analysis

Data analysis

Once we have set up an algorithm which realizes importance sampling Monte Carlo, for example through the use of Markov chains (i.e. generates a Markov Chain which has the desired distribution as an equilibrium distribution), it is possible to generate a sample of field configurations $\phi_i(x)$ according to the probability distribution $e^{-S_E}/\mathcal{Z}$. We will focus on the algorithms later.

Suppose that a sequence of configurations $\phi_1(x), \phi_2(x), \dots, \phi_N(x)$ have been generated, in equilibrium. For each of these configurations, one can compute every desired function of the field, obtaining a sample $\mathcal{O}_1(x), \mathcal{O}_2(x), \dots, \mathcal{O}_N(x)$, where of course $\mathcal{O}_i(x) \equiv \mathcal{O}(\phi_i(x))$. Each $\mathcal{O}_i(x)$ is a random variable and all of these variables have the same expectation value and variance:

$$\langle \mathcal{O}_i \rangle = \langle \mathcal{O} \rangle_{\text{PI}} \quad \sigma_{\mathcal{O}_i}^2 = \langle (\mathcal{O}_i - \langle \mathcal{O}_i \rangle)^2 \rangle = \sigma_{\mathcal{O}}^2, \quad (24)$$

where $\langle \mathcal{O} \rangle_{\text{PI}}$ is the path-integral expectation value. These expectation values have to be thought as averages over an infinite number of replica of N samples. This could be done either considering an infinite number of Markov chains with different initial configurations and sequences, picking up N samples from each configuration, or a single long chain and take an infinite number of (infinitely separated) N samples along the chain.

Let's consider N samples $\mathcal{O}_1, \dots, \mathcal{O}_N$. The mean of a sample, which is

$$\bar{\mathcal{O}} = \sum_{i=1}^N \mathcal{O}_i, \quad (25)$$

is itself a random variable, with mean and variance given by

$$\langle \bar{\mathcal{O}} \rangle = \langle \mathcal{O} \rangle \quad \sigma_{\bar{\mathcal{O}}}^2 = \langle (\bar{\mathcal{O}} - \langle \mathcal{O} \rangle)^2 \rangle = \langle \bar{\mathcal{O}}^2 \rangle - \langle \mathcal{O} \rangle^2, \quad (26)$$

where $\langle \mathcal{O} \rangle \equiv \langle \mathcal{O} \rangle_{\text{PI}}$.

Correlated and uncorrelated data

Let's first consider the case of *correlated samples*, i.e.

$$\langle \mathcal{O}_i \mathcal{O}_j \rangle = \langle \mathcal{O}_i \rangle \langle \mathcal{O}_j \rangle = \langle \mathcal{O} \rangle^2, \quad i \neq j. \quad (27)$$

Then, it is easy to show that

$$\sigma_{\bar{\mathcal{O}}}^2 = \frac{1}{N} \sigma_{\mathcal{O}}^2. \quad (28)$$

In actual simulations, where one has access to a limited number of samples, *stochastic estimators* obtained from data are used for the mean and its errors. In particular, for uncorrelated samples:

$$\bar{\mathcal{O}} = \sum_{i=1}^N \mathcal{O}_i \quad \Delta_{\mathcal{O}}^2 = \frac{1}{N} \Delta_{\mathcal{O}}^2, \quad (29)$$

with

$$\Delta_{\mathcal{O}}^2 = \frac{1}{N-1} \sum_{i=1}^N (\mathcal{O}_i - \bar{\mathcal{O}})^2 \quad (30)$$

are used to estimate the mean and its error.

In case of *correlated samples*, a relevant case for Markov chains, one has

$$\langle \mathcal{O}_i \mathcal{O}_j \rangle \neq \langle \mathcal{O}_i \rangle \langle \mathcal{O}_j \rangle. \quad (31)$$

In this case, it can be shown that

$$\sigma_{\mathcal{O}}^2 = \frac{\sigma_{\mathcal{O}}^2}{N} 2\tau_{\text{int}}^{\mathcal{O}}, \quad \tau_{\text{int}} = \frac{1}{2} \left[1 + 2 \sum_{t=1}^{N-1} \frac{\Gamma_{\mathcal{O}}(t)}{\Gamma_{\mathcal{O}}(0)} \right], \quad (32)$$

where τ_{int} is the *integrated auto-correlation time*, whose expression contains $\Gamma_{\mathcal{O}}(t)$, known as *auto-correlation function*, defined as

$$\Gamma_{\mathcal{O}}(t) \equiv \langle \mathcal{O}_i \mathcal{O}_{i+t} \rangle - \langle \mathcal{O} \rangle^2, \quad (33)$$

and of course $\Gamma_{\mathcal{O}}(0) = \sigma_{\mathcal{O}}^2$. We will see how to compute the auto-correlation function in actual simulation later.

It is clear that the correlation reduces the number of independent sample from N to $\frac{N}{2\tau_{\text{int}}}$. In order to obtain - out of these correlated samples - a new set of uncorrelated samples, one can either skip measurements (measure observables every $2\tau_{\text{int}}$ steps instead of each step) or use the method of *binning*. This consists in building *binned* counterparts out of the original sample $\mathcal{O}_1, \dots, \mathcal{O}_N$, in the following manner:

$$\mathcal{O}_k^B = \frac{1}{B} \sum_{i=1}^B \mathcal{O}_{(k-1)B+i}, \quad k = 1, 2, \dots, N_B, \quad (34)$$

with $N_B = N/B$ being the number of binned samples. If the bin dimension is such that $B \geq 2\tau_{\text{int}}^{\mathcal{O}}$, the samples $\mathcal{O}_1^B, \dots, \mathcal{O}_{N_B}^B$ are expected to be uncorrelated and the error can be computed with the formula for uncorrelated samples. If $B < 2\tau_{\text{int}}$, the error is underestimated.

We will investigate that by means of numerical simulation.

Jackknife method

Another important issue to be discussed is related to the computation of mean and error of functions of expectation values, which we will encounter in the analysis of data of our numerical simulations (for example *susceptibility* and *binder cumulant*, as we will see). In this context, we refer to uncorrelated samples for the observables of interest.

We would like to estimate mean and error of quantities of the form

$$F(\langle a \rangle, \langle b \rangle, \dots), \quad (35)$$

where a and b are *primary* observables. In fact, it is not an easy task to stochastically estimate the error of such functions because one would have to compute difficult derivatives and covariances. For example, the error associated with $F(\langle a \rangle, \langle b \rangle)$ is

$$\sigma_F^2 = \left[\frac{\partial F}{\partial a} \Big|_{\bar{a}, \bar{b}} \right]^2 \sigma_{\bar{a}}^2 + \left[\frac{\partial F}{\partial b} \Big|_{\bar{a}, \bar{b}} \right]^2 \sigma_{\bar{b}}^2 + 2 \left[\frac{\partial F}{\partial a} \Big|_{\bar{a}, \bar{b}} \right] \left[\frac{\partial F}{\partial b} \Big|_{\bar{a}, \bar{b}} \right] \text{cov}(\bar{a}, \bar{b}). \quad (36)$$

At this point, a very powerful method - called *jackknife method* - comes in handy to estimate errors, automatically computing all the relevant derivatives and covariances.

Given a set of samples $a_1, \dots, a_N$, we define the *jackknife samples* as:

$$a_k^J \equiv \frac{1}{N-1} \sum_{i=1, i \neq k}^N a_i = \bar{a} - \frac{a_k - \bar{a}}{N-1}, \quad (37)$$

where $\bar{a} = \frac{1}{N} \sum_{i=1}^N a_i$. For the function F , we can define the corresponding jackknife samples as

$$F_k^J \equiv F(a_k^J, b_k^J, \dots). \quad (38)$$

A stochastic estimate for $F(\langle a \rangle, \langle b \rangle, \dots)$ is given by

$$\bar{F}^J = \bar{F}, \quad (39)$$

while the error can be estimated by

$$(\Delta_F^J)^2 = \frac{N-1}{N} \sum_{k=1}^N (F_k^J - \bar{F}^J)^2, \quad (40)$$

which can be proved to be equal to $(\Delta_F)^2$, the stochastic estimate of the error computed throughout derivatives and covariances.

2 Algorithms

In this section I will present the algorithms used to perform numerical simulation of the scalar field theory ϕ^4 : Metropolis and Hybrid Monte Carlo.

2.1 Metropolis

As previously mentioned, Metropolis algorithm is an efficient way to realize *detailed balance*; therefore, it can be used to construct a Markov chain which approaches a desired probability distribution at long times.

It consists of the following steps:

1. after the initial state of the system E_j has been given, a new state E_k is proposed, with transition probability Q_{jk} satisfying the *micro-reversibility* condition

$$Q_{jk} = Q_{kj}. \quad (41)$$

2. In order to generate a Markov chain with the desired equilibrium probability distribution

$$P_i = \frac{R_i}{\sum_j R_j} \quad (42)$$

the proposed configuration E_k is accepted with the following *acceptance probability*:

$$A_{jk} = \begin{cases} 1 & \frac{R_k}{R_j} > 1, \\ \frac{R_k}{R_j} & \frac{R_k}{R_j} \leq 1. \end{cases} \quad (43)$$

If the proposed configuration is accepted, then the new state of the sequence will be E_k . Otherwise, the new configuration is given by E_j again.

Let's take a look at the algorithm in the specific case of the scalar field theory ϕ^4 ; it is defined by the following steps:

1. an initial random field configuration $\phi_0(x)$ is generated.
2. Given the current field configuration at Markovian time n $\phi_n(x)$, a new configuration is proposed in this way:

$$\phi'_n(x) = \phi_n(x) + \delta_{x,x'} \Delta(r - 0.5) \quad (44)$$

where $\phi'_n(x)$ differs from $\phi_n(x)$ only at space-time point x' , r is a real random number uniformly distributed in $[0, 1)$ and Δ is a parameter controlling the variation of the field at location x' (usually $0 < \Delta < 1$).

3. The proposed configuration $\phi'_n(x)$ is accepted with an acceptance probability

$$P(\phi \rightarrow \phi') = \min[1, e^{-\Delta S}], \quad \Delta S = S[\phi'] - S[\phi] \quad (45)$$

If the proposed configuration is accepted, then $\phi_{n+1}(x) = \phi'_n(x)$, $\forall x$. Otherwise, $\phi_{n+1}(x) = \phi_n(x)$, $\forall x$.

4. 2. and 3. has to be repeated for every x' in the lattice. Once each x' has been considered, it's called *sweep*.

These are the basic steps of Metropolis algorithm. After each sweep, observables can be computed by adopting the field configuration correspondent to the current Markovian time. Before properly sampling, however, the Markov chain should reach its equilibrium configuration: that is why a certain number of field configurations is skipped before actually measuring observables.

2.2 Hybrid Monte Carlo

Metropolis algorithm updates the field one space-time point at a time. This kind of algorithms performing *local* updates are not ideal for generic systems, basically for two reasons.

First, changing only locally the value of the field has the consequence that configuration space is explored quite slowly for systems with high-dimensionality. Second, local updates are very inefficient in the case of *non-local* interactions. This can be computationally very expensive and a scheme implementing *global* updates of the field - i.e. the field is updated over the whole lattice - before the accept/reject step is best-suited in this case.

A very powerful global update scheme is Hybrid Monte Carlo (HMC) algorithm.

Hamiltonian formulation

To start with, let's write the path-integral expectation value of an observable as:

$$\langle O \rangle = \frac{1}{Z_H} \int D\pi D\phi e^{-H[\pi, \phi]} O[\phi], \quad (46)$$

where an auxiliary field $\pi(x)$ has been introduced and

$$Z_H = \int D\pi D\phi e^{-H[\pi, \phi]}, \quad (47)$$

where the *Hamiltonian* has been defined as

$$H[\pi, \phi] = \frac{1}{2} \sum_{x \in \Lambda} \pi(x)^2 + S[\phi]. \quad (48)$$

Also, π and ϕ are now considered as functions of a simulation time τ , so that $\pi(x) \rightarrow \pi(\tau, x)$ and $\phi(x) \rightarrow \phi(\tau, x)$. Within this context, the auxiliary field is the conjugate momentum of the field $\phi(\tau, x)$ with respect to the Hamiltonian; therefore, fields and momenta evolve along the simulation time τ according to the *Hamilton equations*, often referred to as *Molecular Dynamics* (MD) *equations*:

$$\frac{d\phi(\tau, x)}{d\tau} = \frac{\delta H[\pi, \phi]}{\delta \pi(\tau, x)} = \pi(\tau, x) \quad (49)$$

$$\frac{d\pi(\tau, x)}{d\tau} = -\frac{\delta H[\pi, \phi]}{\delta \phi(\tau, x)} = -\frac{\delta S[\phi]}{\delta \phi(\tau, x)} \quad (50)$$

Since these equations are satisfied, it follows that $\frac{dH[\pi, \phi]}{d\tau} = 0$, i.e. the Hamiltonian is a constant of motion along MD trajectories

Ideal HMC algorithm

Although some modifications are necessary in practice, it is useful to show an *ideal* version of the HMC algorithm, in order to exhibit the key ingredients of the theory reformulation in terms of π and ϕ . The ideal algorithm consists in the following steps.

1. A random momentum field π_0 is sampled according to the probability distribution

$$P_G = \frac{1}{Z_\pi} e^{-\frac{1}{2} \sum_{x \in \Lambda} \pi(x)^2}. \quad (51)$$

2. Given such $\pi_0(x)$ and a value for $\phi_0(x)$ (any random value if we are at the beginning of the chain, the field at the previous step otherwise), these are the initial conditions for the MD equations:

$$(\pi(\tau = 0, x), \phi(\tau = 0, x)) \equiv (\pi_0, \phi_0). \quad (52)$$

Provided with these initial conditions, the MD equations are evolved for a time τ_0 , so that new fields are obtained:

$$(\pi_0, \phi_0) \xrightarrow{\text{MD}} (\pi(\tau_0, x), \phi(\tau_0, x)) \quad (53)$$

3. By alternating 1. and 2., an ergodic algorithm which has the desired probability distribution as fixed point is obtained.

It is important to recall that the MD evolution preserves the Hamiltonian of the system, but also the measure of the phase space (*Liouville's theorem*):

$$H(\pi_0, \phi_0) = H(\pi(\tau_0), \phi(\tau_0)) \quad \text{and} \quad D\pi D\phi = D\pi_0 D\phi_0. \quad (54)$$

Leapfrog scheme and Accept-Reject step

The previously described algorithm is ideal because, in most cases, the MD equations cannot be solved exactly: a discrete integration scheme is necessary, such as the *Leapfrog scheme*. To define the integrator, first one has to divide the time interval in $[0, \tau]$ in N_0 steps of size $\delta\tau = \tau_0/N_0$. The solution given by the integrator should be exact in the limit $N \rightarrow \infty$, i.e. $\delta\tau \rightarrow 0$. Let's consider the Taylor expansion of the solutions of MD equations:

$$\pi(\tau + \delta\tau) = \pi(\tau) + \delta\tau \dot{\pi}(\tau) + O(\delta\tau^2) = \pi(\tau) - \delta\tau \mathcal{F}(\tau) + O(\delta\tau^2) \quad (55)$$

$$\phi(\tau + \delta\tau) = \phi(\tau) + \delta\tau \dot{\phi}(\tau) + O(\delta\tau^2) = \phi(\tau) + \delta\tau \pi(\tau) + O(\delta\tau^2) \quad (56)$$

where $\phi(\tau) \equiv \phi(\tau, x)$ (same for π) and

$$\mathcal{F}(\tau, x) \equiv \left. \frac{\delta \mathcal{S}}{\delta \phi} \right|_{\phi(x) \equiv \phi(\tau, x)}. \quad (57)$$

In order to describe the evolution of ϕ and π (up to corrections of order $O(\delta\tau^2)$), we introduce the elementary update steps:

$$\begin{bmatrix} \pi(\tau + \delta\tau) \\ \phi(\tau) \end{bmatrix} = I_\pi(\delta\tau) \begin{bmatrix} \pi(\tau) \\ \phi(\tau) \end{bmatrix} = \begin{bmatrix} \pi(\tau) - \delta\tau \mathcal{F}(\tau) \\ \phi(\tau) \end{bmatrix}, \quad (58)$$

$$\begin{bmatrix} \pi(\tau) \\ \phi(\tau + \delta\tau) \end{bmatrix} = I_\phi(\delta\tau) \begin{bmatrix} \pi(\tau) \\ \phi(\tau) \end{bmatrix} = \begin{bmatrix} \pi(\tau) \\ \phi(\tau) + \delta\tau \pi(\tau) \end{bmatrix}. \quad (59)$$

Then, we define the LPF (Leapfrog) integrator as

$$I_{\text{LPF}}(N_0, \delta\tau) \equiv \left[I_\pi \left(\frac{\delta\tau}{2} \right) I_\phi(\delta\tau) I_\pi \left(\frac{\delta\tau}{2} \right) \right]^{N_0} \quad (60)$$

where some initial conditions $\pi(0)$ and $\phi(0)$ must be provided.

The LPF - as we will study through numerical simulations - maintains important properties of MD evolution, in particular is time-reversible and preserves the phase-space measure. For these reasons, it is called a *symplectic* integrator. Being time-reversible means

$$I_{\text{LPF}}(N_0, -\delta\tau) I_{\text{LPF}}(N_0, \delta\tau) = 1 \quad \forall N_0, \delta\tau, \quad (61)$$

while preserving the phase-space measure means

$$D\pi(\tau_0) D\phi(\tau_0) = D\pi D\phi \quad \text{with} \quad \begin{bmatrix} \pi(\tau_0) \\ \phi(\tau_0) \end{bmatrix} = I_{\text{LPF}}(N_0, \tau) \begin{bmatrix} \pi \\ \phi \end{bmatrix}. \quad (62)$$

Despite being symplectic, the LPF integrator does not conserve the Hamiltonian. However, it conserve a quantity called *shadow Hamiltonian* $\tilde{H}$ such that

$$H(\pi, \phi) - \tilde{H}(\pi, \phi) = O(\delta\tau^2). \quad (63)$$

It can be shown that along a LPF trajectory, the error on H satisfies

$$|H(\pi, \phi)| \leq O(\delta\tau^2). \quad (64)$$

Since the LPF integrator does not conserve the Hamiltonian, the ideal HMC algorithm described before can not be used in that form. This can be solved easily in the following way. After solving the MD equations by means of LPF integrator, an Accept-Reject step is introduced. In particular, the proposed configuration obtained after the LPF integration $(\pi(\tau_0), \phi(\tau_0))$ is accepted with probability

$$P_A = \min[1, e^{-\Delta H}] \quad \Delta H = H(\pi(\tau_0), \phi(\tau_0)) - H(\pi_0, \phi_0), \quad (65)$$

otherwise ϕ_0 is the next field of the chain. In the limit $N_0 \rightarrow \infty$, we have $\Delta H = 0$ and the new configurations are automatically accepted.

3 Numerical results

In order to discuss the results obtained, I am now going to present in this section the building blocks of the simulation program.

The potential adopted here is a quartic self-interaction

$$V(\phi) = \frac{g_0}{4!} \phi^4. \quad (66)$$

The action has been redefined as

$$\mathcal{S}[\phi'] = \sum_{x \in \Lambda} \left[-2\kappa \sum_{\mu=0}^d \phi'(x) \phi'(x + \hat{\mu}a) + \phi'(x)^2 + \lambda(\phi'(x)^2 - 1)^2 \right] \quad (67)$$

where the original field ϕ is related to the new one by the relation

$$\phi = \frac{2\kappa}{a^{\frac{d+1}{2}}} \phi'. \quad (68)$$

We will call ϕ the field ϕ' , from now on. Moreover, m and g_0 are related to the new parameters κ and λ by

$$m^2 = \frac{1}{a^2} \left(\frac{1-2\lambda}{\kappa} - 2(d+1) \right), \quad g_0 = \frac{6\lambda}{a^{3-d\kappa^2}}. \quad (69)$$

In the program, the routine which computes the action is shown below.

```
double action(act_params_t *apars)
{
    int i,mu;
    double phin,S,phi2;
    double kappa =apars->kappa;    /* Get the "kappa" parameter */
    double lambda=apars->lambda;    /* Get the "lambda" parameter */

    S=0;

    /* Loop over all sites */
    for (i=0;i<V;i++)
    {
        /* Sum over neighbors in positive direction */
        phin=0;
        for (mu=0;mu<D;mu++) phin+=phi[hop[i][mu]];

        phi2=phi[i]*phi[i];
        S+=-2*kappa*phin*phi[i]+phi2+lambda*(phi2-1.0)*(phi2-1.0);
    }

    return S;
}
```

3.1 The ϕ^4 -theory: Metropolis implementation

Here I show the routine implementing Metropolis algorithm.

```
double metropolis(act_params_t *apars, metro_params_t *mpars)
{
    int i, acc;
    static double rand[2];
    double S, S_prime, deltaS, tmp;
    double delta=mpars->delta;      /* Get the "delta" parameter */
    acc=0;

    /* Do a sweep over the lattice */
    for(i=0; i<V; i++)
    {
        ranlxd(rand, 2);             /* Compute two random numbers */
        S = action(apars);           /* Get the current action */
        tmp = phi[i];                /* Save the field at location i */
        phi[i] += delta*(rand[0]-0.5); /* Update the field at location i */
        S_prime = action(apars);     /* Get the updated action */
        deltaS = S_prime - S;        /* Compute the variation of the action */

        /* Metropolis accept-reject step */
        if (rand[1] < exp(-deltaS)) {acc++;} /* Accepted */
        else {phi[i] = tmp;}             /* Rejected */
    }

    return acc;      /* Return the number of accepted configurations */
}
```

Part I: Computing observables

Before introducing our observables, let's define a function of the field as

$$M \equiv \sum_{x \in \Lambda} \phi(x) \quad (70)$$

The observables we are interested in are functions of M . In particular, we shall consider

- *Absolute magnetization:* $|m| = \frac{1}{V} \langle |M| \rangle$
- *Square magnetization:* $m^2 = \frac{1}{V} \langle M^2 \rangle$
- *Susceptibility:* $\chi = \frac{1}{V} (\langle M^2 \rangle - \langle |M| \rangle^2)$
- *Binder cumulant:* $B = \frac{\langle M^4 \rangle}{\langle M \rangle^2}$

Here I report the code used to compute M .

```
double M()
{
    mag=0.;

    /* Sum of the field's values over all lattice locations */
    for(i=0;i<V;i++) {mag+=phi[i];}

    return mag;
}
```

Since all the observables of interest are functions of the absolute value of M and of M powers - M^2 and M^4 in particular - it is convenient to write separate functions for $|M|, M^2, M^4$:

```
double Mabs()
{
    return fabs(M());          /* Compute the absolute value of M */
}

double Mtwo()
{
    return pow(M(),2);         /* Compute the square of M */
}

double Mfour()
{
    return pow(M(),4);         /* Compute the fourth power of M */
}
```

For this study, the following parameters has been used:

$$D = 3, \quad L = 4, \quad \lambda = 1.145, \quad \kappa = 0.15, \quad \Delta = 0.25,$$

with $n_{\text{therm}}=10^5$ and $n_{\text{sweep}}=10^8$, where n_{therm} and n_{sweep} are respectively the number of thermalization steps and lattice sweeps after thermalization. In the first study, the following results had to be confirmed:

$$|m| = 0.19246(54)$$

$$m^2 = 3.576(18)$$

$$\chi = 1.2053(57)$$

$$B = 2.5897(76)$$

Here is reported a part of the main program, which shows the routines which perform thermalization steps and sweeps after thermalization.

```

/* thermalization steps */
for(i=0;i<metro_params.ntherm;i++)
{
    metropolis(&act_params,&metro_params);
}

/* sweeps after thermalization, computation of observables */
for(i=0;i<metro_params.nsweep/metro_params.naccu;i++)
{
    m=0.;
    sm=0.;
    qm=0.;
    for(j=0;j<metro_params.naccu;j++)
    {
        m+=Mabs();
        sm+=Mtwo();
        qm+=Mfour();
        metropolis(&act_params,&metro_params);
    }
    printf("%d %e %e %e \n",i+1,m/(metro_params.naccu),sm/(metro_params.naccu),
        qm/(metro_params.naccu));
}

```

In particular, I have generated data just for $|M|, M^2, M^4$ since, as previously mentioned, all the observables which we are interested in are functions of these. The output of these data has been redirected to a text file; then, I have used Python (in Jupyter Notebook) in order to analyze and visualize data. Let's see how. First, from the simulation program I have obtained the following samples:

$$|M|_1, \dots, |M|_N \quad (M^2)_1, \dots, (M^2)_N \quad (M^4)_1, \dots, (M^4)_N, \quad (71)$$

collected respectively in the arrays `m`, `m2` and `m4`. Having the samples, the mean and its error can be computed. Here I have used the Python library `numpy`, in particular the routines `numpy.mean` and `numpy.std`, in order to compute the mean and its error. After collecting the samples in the arrays `m`, `m2` and `m4`, we can work out all the relevant observables.

Let's first show the jackknife routine, which has been used to compute errors (especially in the case of susceptibility and binder cumulant).

```
def jackknife(x):
    ajn = np.array([])
    for i in range (0,N):
        ajn = np.append(ajn, np.mean(x) - (x[i] - np.mean(x))/(N-1))
    return ajn
```

This routines takes an array - which represents the original samples - as input and returns the correspondent jackknife samples.

Note that in the case of $|m|$ and m^2 the error can be computed using the formula (30). The results have been confirmed using the jackknife method.

Absolute magnetization

```
M_mean = np.mean(m/V)
ErrM = np.std(m/V)/(np.sqrt(N))
```

$$|m| = 0.19225(51)$$

and with jackknife:

```
mjn = jackknife(m/V)
M_jn_mean = np.mean(mjn)
ErrM_jn = np.sqrt((N-1)/N*np.sum((mjn-np.mean(mjn))**2))
```

$$|m| = 0.19225(51)$$

Square magnetization

```
M_mean = np.mean(m2/V)
ErrM = np.std(m2/V)/(np.sqrt(N))
```

$$m^2 = 3.562(17)$$

and with jackknife:

```
m2jn=jackknife(m2/V)
M2_jn_mean = np.mean(m2jn)
ErrM2_jn = np.sqrt((N-1)/N*np.sum((m2jn-np.mean(m2jn))**2))
```

$$m^2 = 3.562(17)$$

In the case of susceptibility and binder cumulant, the error can be computed with formula (36), which involves derivatives and covariances. Even in this case, the results with the ones obtained using jackknife method. Let's see.

Susceptibility

Having already $|m|$ and m^2 , the mean value is easily computed as

$$\chi = \frac{1}{V}(\langle M^2 \rangle - \langle |M| \rangle^2) = m^2 - |m|V \quad (72)$$

Therefore:

```
Susc = np.mean(m2/V) - V*np.mean(m/V)**2
```

The jackknife samples corresponding to χ are given by

```
suscjn=m2jn-V*(mjn**2)
```

The jackknife mean `np.mean(suscjn)` is obviously equal to `Susc`. The error is

```
ErrSusc_jn = np.sqrt((N-1)/N*np.sum((susc-np.mean(susc))**2))
```

Finally we have:

$$\chi = 1.1968(53)$$

Note that, computing the error with the formula (36)

```
covM_M2 = np.cov(m/V,m2/V)[0,1]/N
```

```
ErrSusc=np.sqrt((ErrM**2)*((M_mean*2*V)**2) + ErrM2**2 - 4*V*(M_mean)*covM_M2)
```

confirms the jackknife result.

Binder cumulant

In this case we have:

```
Bind = (np.mean(m4))/(np.mean(m2))**2
```

The jackknife samples are given by

```
bindjn = m4jn/(V**2 * m2jn**2)
```

having defined the jackknife samples correspondent to M^4 as

```
m4jn = jackknife(m4)
```

The jackknife mean of `bindjn` is equal to `Bind` as should be. The error is

```
ErrBind_jn = np.sqrt((N-1)/N*np.sum((bind-np.mean(bind))**2))
```

Thus:

$$B = 2.5857(74)$$

and note that, even in this case, the error is the same using formula (36)

```
ErrBind=np.sqrt(1/(y**4)*(sigmax**2) + 4*(x**2)/(y**6)*(sigmay**2) - 4*x/(y**5)*sigmaxy)
```

with `x = m4_mean`, `y = m2_mean`, `sigmax = errm4`, `sigmay = errm2` and `sigmaxy = np.cov(m/V,m2/V)[0,1]/N`

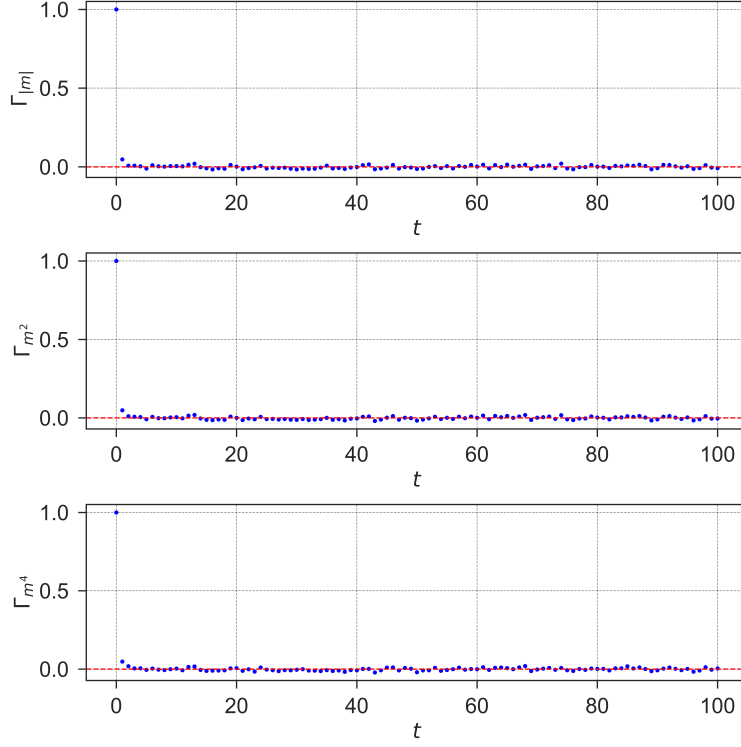


Figure 1: Autocorrelation function for $|m|, m^2, m^4$.

To show that the samples are actually uncorrelated with $\text{naccu}=10^4$, we have computed the autocorrelation function with the following routine:

```
def Gamma(x):
    x_mean=np.mean(x)
    gamma=np.array([])
    for j in range(0,N):
        sum=0
        for i in range(0,N-j):
            sum+=(x[i]-x_mean)*(x[i+j]-x_mean)
        sum/=(N-j)
        gamma=np.append(gamma,sum)
    return gamma
```

which takes an array of values - which in our case is the sequence of samples $\mathcal{O}_1, \dots, \mathcal{O}_N$ - and returns an array which has $\Gamma_{\mathcal{O}}(t)$ as t -th element.

Part II: Autocorrelation

In the second part of the Metropolis study I computed the auto-correlation function for the observable m^2 , $\Gamma_{m^2}(t)$ by keeping fixed the parameters

$$D = 3, \quad L = 4, \quad \lambda = 1.145, \quad \kappa = 0.15,$$

but varying the parameter Δ . In particular, the simulations have been performed with

$$\Delta = 0.1, 0.25, 0.4.$$

Note also that in this case the value for `naccu` should be chosen such that some auto-correlation could be detected. A value `naccu`= 100 has been used here.

The result (the first 100 values) is shown in Fig.2. The points obtained by the simulations are very well fit by a one-parameter τ exponential function

$$f(x, \tau) = \exp(-x/\tau) \tag{73}$$

The τ can be interpreted as a *time-constant*, telling us how rapidly the exponential decreases. In particular, after a time τ it decreases a factor $1/e$, $\simeq 36.8\%$ of its initial value.

I obtained - for $\Delta = 0.1, 0.25, 0.4$ - the values $\tau \simeq 48.12, 8.10, 3.43$ respectively. This means that, by increasing the parameter Δ , the autocorrelation decays more quickly, as it is clear from Fig.2.

In another part of the study - still on $\Gamma_{m^2}(t)$ - simulations have been performed by keeping fixed all the previous parameters, with also $\Delta = 0.25$ (fixed as well), but varying the bin-size. I have run different simulations, but the effect is visible only on those with smaller bin-sizes - otherwise the autocorrelation function decays too much rapidly. In this case, the tests have been performed for bin-sizes of 100, 500, 1000, obtaining $\tau \simeq 8.10, 1.92, 1.13$. The result is shown in Fig.3. As expected, increasing the bin-size, leads to a more rapid decreasing of the autocorrelation function.

For the curve fitting I used the Python function `curve_fit` from the module `optimize` of the library `scipy`.

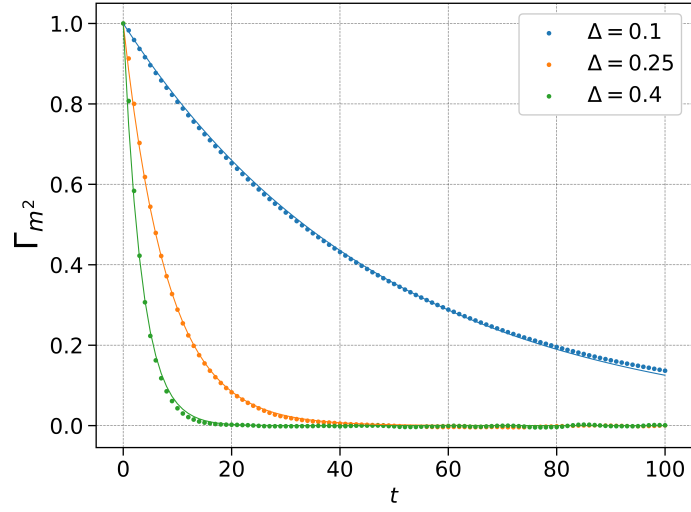


Figure 2: Normalized m^2 -autocorrelation function for different values of Δ .

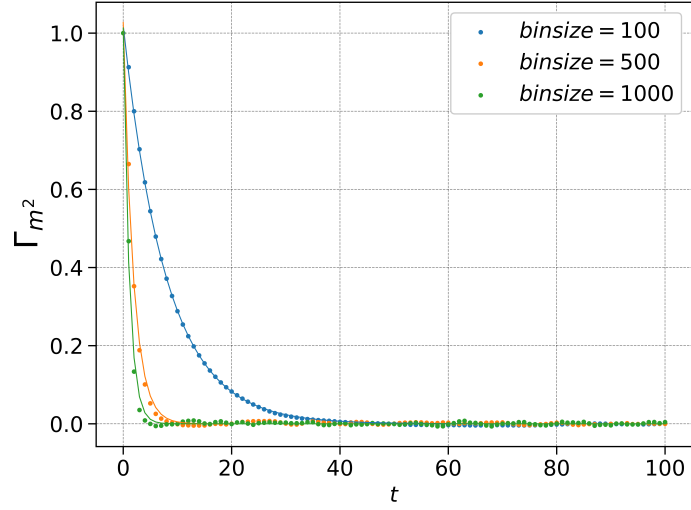


Figure 3: Normalized m^2 -autocorrelation function for different values of the bin-size.

3.2 The ϕ^4 -theory: Hybrid Monte Carlo implementation

Here I show the routine implementing Hybrid Monte Carlo algorithm.

```
double hmc(act_params_t *apars, metro_params_t *mpars, double deltat, int N)
{
    int i;
    static double rand[1];
    double H, H_prime, deltaH;
    double phi_old[V];
    double x1[V], x2[V];
    double acc;

    acc=0;
    ranlxd(rand,1); /* Compute one random number */
    ranlxd(x1,V); /* Compute V random numbers */
    ranlxd(x2,V); /* Compute V random numbers */

    /* Use Box-Muller formula */
    for(i=0;i<V;i++){
        mom[i] = (sqrt(-2*log(1-x1[i]))) * cos(2*M_PI*(1- x2[i]));
    }

    H = hamiltonian(apars); /* Compute Hamiltonian */

    /* Save old phiold */
    for(i=0;i<V;i++){
        phi_old[i] = phi[i];
    }

    Leapfrog(apars,deltat,N); /* Leapfrog step */

    H_prime = hamiltonian(apars); /* Compute Hamiltonian */
    deltaH = H_prime - H; /* Compute Delta H */

    /* Accept-Reject step */
    if (rand[0]<exp(-deltaH)) {acc++;} /* Accepted */
    else
    {
        for(i=0;i<V;i++) {phi[i] = phi_old[i];} /* Rejected */
    }

    return deltaH; /* Returns Delta H */
}
```

The LPF integrator implementation is shown below.

```
void move_pi(act_params_t *apars, double deltat)
{
    double F[V];
    double kappa=apars->kappa;
    double lambda=apars->lambda;
    double phin,phi2;
    int i,mu;

    for(i=0;i<V;i++){

        phin=0;
        for (mu=0;mu<D;mu++) phin+=(phi[hop[i][mu]] + phi[hop[i][D+mu]]);
        phi2=phi[i]*phi[i];
        F[i]=-2*kappa*phin + 2*phi[i] + 4*lambda*(phi2-1.0)*phi[i];
    }

    for(i=0;i<V;i++){
        mom[i]+=(-1)*deltat*F[i];
    }
}

void move_phi(act_params_t *apars, double deltat)
{
    int i;

    for(i=0;i<V;i++){
        phi[i]+=deltat*mom[i];
    }
}

void Leapfrog(act_params_t *apars, double deltat, int N)
{
    int i;

    for (i=0;i<N;i++){
        move_pi(apars,0.5*deltat);
        move_phi(apars,deltat);
        move_pi(apars,0.5*deltat);
    }
}
```

Some tests can be made to check the correctness of the implementation.

Reversibility

This check should demonstrate that the Leapfrog integrator is time-reversible. This can be implemented in these steps.

- Starting with a configuration (π, ϕ) and after choosing $\tau_0, N_0, \delta\tau$, integrate the MD equations using the Leapfrog integrator, obtaining $(\pi', \phi') = (\pi(\tau_0), \phi(\tau_0))$.
- Flip the momenta. This means $(\pi', \phi') \rightarrow (-\pi', \phi')$.
- Integrate the MD equations, now starting from $(-\pi', \phi') = (-\pi(\tau_0), \phi(\tau_0))$. A new configuration $(-\pi'', \phi'') = (-\pi'(\tau_0), \phi'(\tau_0))$ is obtained.

One should have that $\pi'' = \pi$ and $\phi'' = \phi$. This can be verified measuring

$$\|\phi - \phi''\| = \frac{1}{V} \sum_{x \in \Lambda} |\phi(x) - \phi''(x)|,$$

$$\|\pi - \pi''\| = \frac{1}{V} \sum_{x \in \Lambda} |\pi(x) - \pi''(x)|.$$

One can also look at $\Delta\Delta H = H(\pi'', \phi'') - H(\pi, \phi)$, which should be zero since the error generated in the two trajectories must be equal. Here is how I have implemented the test. After assigning random configurations for `pi` and `phi`:

```
for(i=0;i<V;i++){
    phi_old[i]=phi[i]; /* Save the initial field config. */
    mom_old[i]=mom[i]; /* Save the initial mom config. */
}

h_old=hamiltonian(&act_params); /* Initial Hamiltonian */
Leapfrog(&act_params,dt,N); /* Leapfrog integration */

for(i=0;i<V;i++){mom[i]=-mom[i];} /* Flip momenta */

Leapfrog(&act_params,dt,N); /* Leapfrog integration */

for(i=0;i<V;i++){mom[i]=-mom[i];} /* Flip momenta */

for(i=0;i<V;i++){
    deltaphi+=fabs(phi_old[i]-phi[i]);
    deltamom+=fabs(mom_old[i]-mom[i]);
}

deltaphi/=V; /* Compute the norm of the field difference */
deltamom/=V; /* Compute the norm of the mom difference */

printf("%e %e %e\n", deltaphi, deltamom, hamiltonian(&act_params)-h_old);
```

The following parameters have been used:

$$L/a = 4, \quad D = 3, \quad \kappa = 0.18169, \quad \lambda = 1.3282, \quad \tau_0 = 1 \quad N = 20.$$

I have obtained the following results.

$$||\phi - \phi''|| \simeq 1.09 \times 10^{-16}$$

$$||\pi - \pi''|| \simeq 2.18 \times 10^{-16}$$

$$\Delta\Delta H = 0.$$

The first two numbers can safely be considered to be zero, within round-off errors. Therefore, the reversibility of the algorithm has been checked.

Hamiltonian conservation

As a second check, given the following parameters

$$L/a = 4, \quad D = 3, \quad \kappa = 0.18169, \quad \lambda = 1.3282, \quad \tau_0 = 1$$

we had to verify that $\langle |\Delta H| \rangle = O(\delta\tau^2)$.

To do that, I have generated for five different values of step-size `ntraj`=10⁴ thermalized trajectories and computed $\langle |\Delta H| \rangle$.

Since the relation to be verified was $\langle |\Delta H| \rangle = O(\delta\tau^2)$, it should be also true that $\log_{10}(\langle \Delta H \rangle) \propto 2 \log_{10}(\delta t)$. Therefore, first I obtained the points $(\log_{10}(\delta t), \log_{10}(\langle \Delta H \rangle))$ for the following time-steps:

$$\frac{1}{100}, \frac{1}{80}, \frac{1}{60}, \frac{1}{40}, \frac{1}{20}.$$

Then, I tried to fit them with a linear function:

$$y = a x + b,$$

where of course $x = \log_{10}(\delta t)$ and $y = \log_{10}(\langle \Delta H \rangle)$. We are interested in the a coefficient, which should be 2. In fact, I obtained the following value for a :

$$a = 2.0046808 \pm 0.0000012.$$

Therefore, we have verified the relation. The plot is reported in Fig.4.

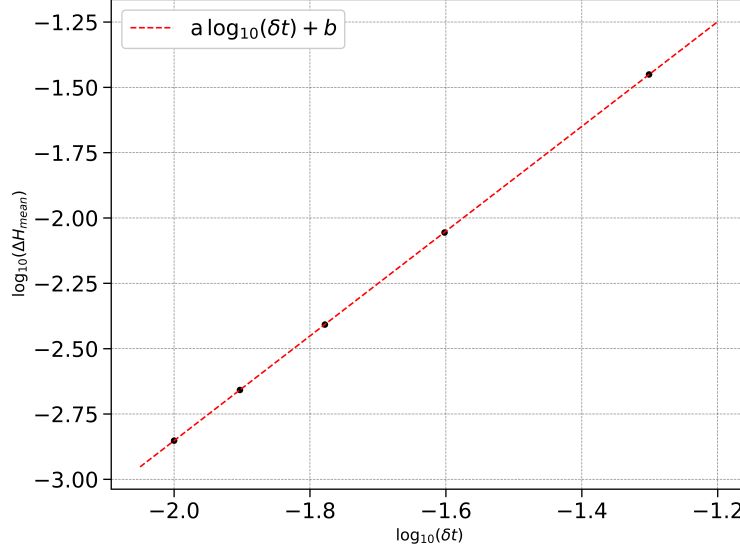


Figure 4: $\log_{10}(\langle \Delta H \rangle)$ as a function of $\log_{10}(\langle \delta t \rangle)$. $H_{mean} \equiv \langle \Delta H \rangle$.

Inexact vs Exact algorithm

In this test, we had to compute m^2 (which is computed as before) with and without the accept-reject step. What do we expect?

With the accept-reject step, the results should be independent on $\delta\tau$; in its absence, the results should show $O(\delta\tau^2)$ corrections with respect to the measurements obtained with the exact algorithm. The two algorithms should give the same results when $\delta\tau \rightarrow 0$.

The following parameters have been used.

$$L/a = 4, \quad D = 3, \quad \kappa = 0.18169, \quad \lambda = 1.3282, \quad \tau_0 = 1,$$

with `ntraj`=100000 thermalized trajectories, and `naccu`=1000 to be sure the measurements are not correlated. I considered, as before, the following size-steps:

$$\frac{1}{100}, \frac{1}{80}, \frac{1}{60}, \frac{1}{40}, \frac{1}{20}.$$

Results are shown in Fig.5.

The first plot shows that the points obtained with the exact algorithms do not depend on the $\delta\tau$, as expected, while the points obtained without the accept-reject step are dependent on $\delta\tau$. It is clear that, as $\delta\tau \rightarrow 0$, they approach to the same result, as it is visible in the figure.

The second plot reports the difference between the measurements obtained with and without the accept-reject step, which should be of order $O(\delta\tau^2)$. As in the case of the Hamiltonian conservation, I took the logarithms (this time I have

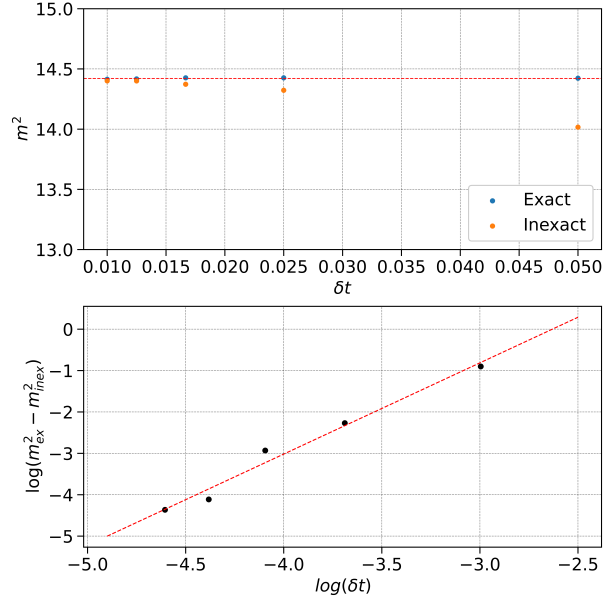


Figure 5: Algorithms with and without accept-reject step.

taken natural logarithms) and tried to fit with a linear function

$$y = a x + b.$$

We expect $a = 2$. I found:

$$a = 2.204 \pm 0.034,$$

which is quite close to the expected value $a = 2$. To be more accurate, one should consider more and more points.

Phase-space measure preservation

The last check performed is related to the area-preserving property of the Leapfrog integrator. In fact, given

$$\Delta H = H(\pi(\tau_0), \phi(\tau_0)) - H(\pi_0, \phi_0),$$

it holds the following relation:

$$\begin{aligned}\langle e^{-\Delta H} \rangle &= \frac{1}{Z_H} \int D\pi_0 D\phi_0 e^{-\Delta H} e^{-H(\pi_0, \phi_0)} \\ &= \frac{1}{Z_H} \int D\pi_0 D\phi_0 e^{-H(\pi(\tau_0), \phi(\tau_0))} \\ &= \frac{1}{Z_H} \int D\pi_0 D\phi_0 e^{-H(\pi(\tau_0), \phi(\tau_0))} \\ &= 1.\end{aligned}$$

Here I show the part of the code I used to generate `ntraj=105` thermalized trajectories, in order to make the measurements. I used a time-step of $\delta t = 0.05$.

```
for(i=0; i<hmc_params.ntraj; i++){
    Exp=0;
    for(j=0; j<metro_params.naccu; j++){
        Exp+=exp(-hmc(&act_params, &metro_params, dt, N));
    }
    printf("%d %e \n", i, Exp/metro_params.naccu);
}
```

The result I have obtained is

$$\langle e^{-\Delta H} \rangle = 1.0000048 \pm 0.0000029.$$

We have obtained the expected result within the errors; therefore, the phase-space measure preservation is confirmed.

4 SSB in ϕ^4 -theory

Classically, the action of the theory - which we have presented before - is invariant under $\mathbb{Z}_2$ transformation: $\phi \rightarrow -\phi$. In case of $m_0^2 > 0$, even the solutions of the EOM are invariant with respect to this transformation: the theory has a symmetry. However, in the case $m_0^2 < 0$, solutions are in general not invariant under $\mathbb{Z}_2$ transformation; however, given a certain solution, the field obtained by applying a $\mathbb{Z}_2$ transformation is still a solution of the EOM.

The corresponding situation in QFT is called *spontaneous symmetry breaking* (SSB) of the $\mathbb{Z}_2$ symmetry. When the *physical mass* of the theory $m_R^2 < 0$, the theory has a non-vanishing expectation value for the vacuum state

$$\langle \phi \rangle \neq 0, \quad (74)$$

despite the action is invariant under $\mathbb{Z}_2$ transformation. Note that the relation between the physical mass and the *bare mass* is

$$m_R = Z_{m_0}(g_0, am_R) \left(m_0 + \frac{c(g_0)}{a} \right), \quad (75)$$

with Z_{m_0} being a renormalization factor diverging logarithmically as the continuum limit ($am_R \rightarrow 0$) is taken.

The SSB breaking is intrinsically non-perturbative and the lattice formulation of QFT is the only known first-principle theory which is able to study it. In our numerical simulations, we wanted to identify - for a specific value of the bare coupling g_0 - the value of m_0 for which m_R . Doing that, one can say where the SSB occurs identifying a *critical point*. Using the parameters of our action, we want to study - for a given λ - the behaviour of some observables as κ changes. We consider in particular:

$$|m|, \quad \frac{m^2}{V}, \quad \chi, \quad B.$$

Note that, however, SSB occurs at infinite volume, i.e. $L \rightarrow \infty$. Given the analogy between the lattice QFT and a classic statistical system, SSB below a certain value of m_R can be interpreted as a phase-transition occurring at a certain temperature. Since we are forced to work in a finite-size lattice, we have to study the phase-transition using *scaling* techniques: the fact that the theory manifests a phase-transition at infinite volume forces the finite-volume observables to behave in a specific way when the volume increases. We have investigated the relevant observables for different value of L/a to identify where the phase-transition occurred. I have performed numerical simulations using the following parameters:

$$L/a = 4, 6, \dots, 14, 16 \quad D = 3, \quad \kappa = 0.15, 0.16, \dots, 0.22, 0.23 \quad \lambda = 1.145$$

Moreover, I have generated for each simulation **ntraj**= 10^5 thermalized trajectories, with a value **naccu**=1000 to be sure that measurements were uncorrelated. The correlation can be easily measured by using the routine for the autocorrelation function which I presented in the Metropolis study.

4.1 Data analysis

In order to analyze where the phase transitions occurred, we can start by taking a look at the *Binder cumulant*

$$B = \frac{\langle M^4 \rangle}{\langle M^2 \rangle^2}.$$

This quantity has interesting features. First, it can be used to distinguish *first order* and *second order* phase transitions. It turns out, in fact, that B behaves differently near first-order and second-order transitions on finite lattice. Therefore, it can be used to determine the order of the phase transition. This cumulant is sometimes defined as

$$U_4 = 1 - \frac{\langle M^4 \rangle}{3\langle M^2 \rangle^2}. \quad (76)$$

- If the transition is second-order, it smoothly varies from its asymptotic value of $U_4 \approx \frac{2}{3}$ (equivalent to $B \approx 1$) when $T \ll T_c$ (in our case $\kappa \gg \kappa_c$) to the value $U_4 \approx 0$ (equivalent to $B \approx 3$) when $T \gg T_c$ (in our case $\kappa \ll \kappa_c$).
- If it is first-order, the cumulant has a minimum which becomes more pronounced as the lattice-size increases.

Another interesting feature, observed in second-order phase transitions, is that U_4 curves for different lattice sizes intersect at a unique point, which identifies the critical temperature T_c of the infinite system (in our case it identifies κ_c).

I have used these features, in this study, to analyze the phase transition. The result is shown in Fig.6, where I have plotted B for different lattice sizes as a function of κ . First of all, it is clear that the phase transition is of the *second order*; in fact, it is clear that it smoothly varies from $B \approx 3$ for $\kappa \ll \kappa_c$ to $B \approx 1$ for $\kappa \gg \kappa_c$. Moreover, the general shape of the Binder cumulant is sigmoidal, it can be very well fit by the hyperbolic tangent. In particular, I fit the data with the following function

$$y = a \tanh(b(x - c)) + d.$$

The interesting parameter is c , which determines the location of the inflection point. At this point, curves corresponding to different lattice sizes should intersect, reporting the location of the critical point.

In particular, obtaining the value of c gives roughly the critical value κ_c . As it is visible in Fig.6, I obtained roughly

$$\kappa_c \approx 0.185. \quad (77)$$

This can be identified as the value at which the phase transition occurs.

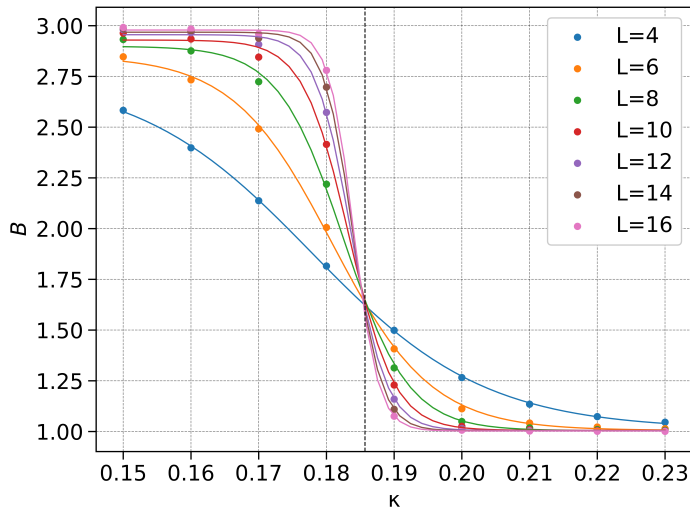


Figure 6: Binder cumulant B as function of κ .

In general, as said before, the finite-volume observables exhibits some characteristic behaviour at the critical parameters as the volume increases. We therefore expect that at the same κ_c also the other observables manifest the phase-transition.

For example, let's first take a look at the absolute magnetization $|m|$. In an Ising system, $|m|$ is close to unity at low temperatures ($T < T_c$) and close to zero at high ones ($T > T_c$). We expect the same behaviour in our case, with $|m|$ close to unity at high κ ($\kappa > \kappa_c$) and close to zero at low κ ($\kappa < \kappa_c$). In fact, this behaviour is visible in Fig.7, which reports the results of the simulation for $|m|$. We observe approximately the same κ_c as before. However, for $\kappa < \kappa_c$, it should be *exactly* zero: the fact that we do not observe it in Fig.7 is a finite-size effect, due to the fact that we are simulating the model in a finite lattice. In fact, we can see that for larger lattice sizes (see for example $L = 16$), the curve exhibits a behaviour closer to the infinite-volume behaviour ($|m| = 0$ for $\kappa < \kappa_c$).

Analogous considerations can be made for the observable m^2/V . The results for this observables are reported in Fig.8.

Let's now focus on the susceptibility χ . From the theory, we know that - by analogy with the Ising model - it should diverge at the critical temperature T_c . At finite size, it exhibits a maximum in correspondence of the critical point. The critical value k_c can be roughly identified by looking at the maxima locations. Moreover, at finite size, the behaviour of this quantity is affected by the size of the lattice. *Finite size scaling* - i.e. observing how the properties vary with the lattice size - provides a useful way to identify some characteristics of the phase transition such as *critical exponents*, which universally describe the behaviour

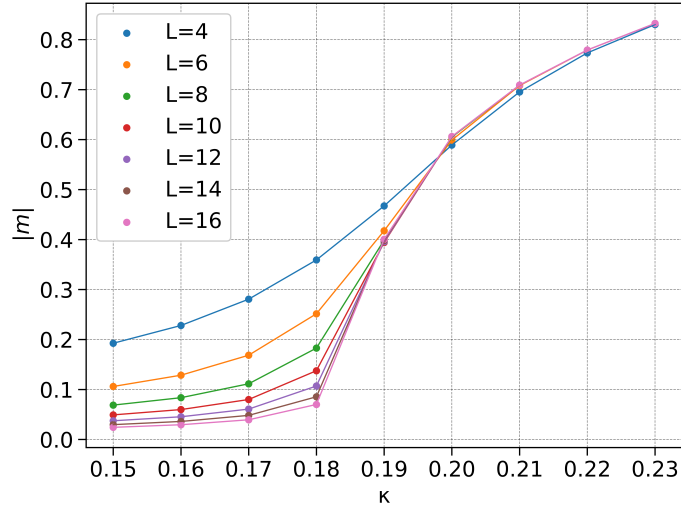


Figure 7: Absolute magnetization $|m|$ as function of κ .

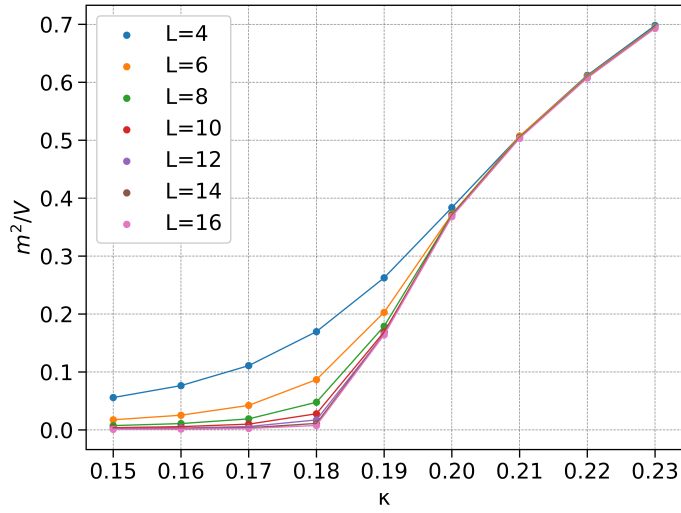


Figure 8: Square magnetization m^2/V as function of κ .

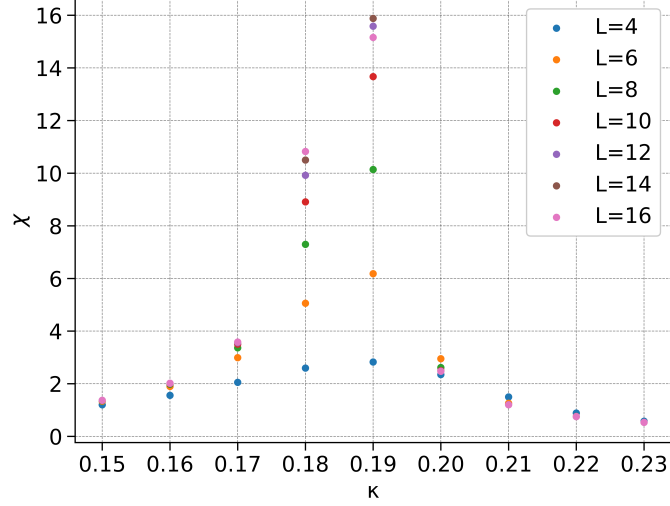


Figure 9: Susceptibility χ as function of κ .

of physical quantities near phase transitions. It turns out that:

$$\max(\chi) \propto L^{\frac{\gamma}{\nu}}.$$

At this point, the goal was fitting the points and find maxima for all the curves, in order to identify k_c . Unfortunately, I was not able to properly fit data using simple polynomial functions. The results of the simulation is shown in Fig.9. However, it is clear from Fig.9 that the maxima are located between $\kappa = 0.18$ and $\kappa = 0.19$; moreover, fitting with a Lorentzian function, I roughly found the maxima location for these curves at $\kappa_c \approx 0.186$, which is very close to the value obtained through the Binder cumulant. Since I was not able to properly fit all the curves, it is difficult to verify the scaling relation of the maxima. However, I managed to fit quite well the curves with $L = 4$, $L = 6$, $L = 8$, $L = 10$ and therefore identify their maxima. Then I tried to fit $\max(\chi)$ as function of L with

$$y = a x^b.$$

The determination of b gives the ratio $\frac{\gamma}{\nu}$, which in this case should be $\frac{\gamma}{\nu} \approx 1.96$. The result of the fit is shown in Fig.10.

I obtained

$$b = 2.029 \pm 0.003.$$

Therefore, I obtained for the ratio of the critical exponents $\frac{\gamma}{\nu} \approx 2$, quite close to the real value $\frac{\gamma}{\nu} \approx 1.96$.

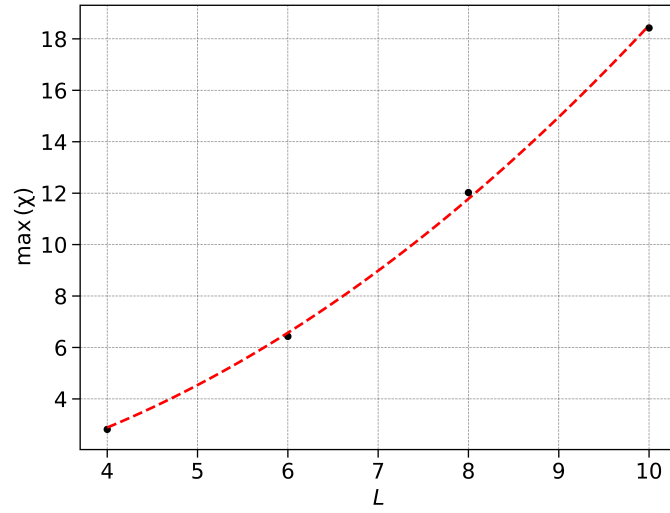


Figure 10: Maxima of χ as a function of L .

Error analysis

I would like to note that, in the present study, I did not plot the error bars because they were almost invisible since the errors on the observable measurements were small. The errors can be computed exactly in the same way as I did in the Metropolis study, adopting the jackknife method (and therefore using the same numerical routine) once a reasonable value for bin-size (`naccu`) - in order to have uncorrelated measurements - has been chosen. The error analysis is analogous to the one made for Metropolis.